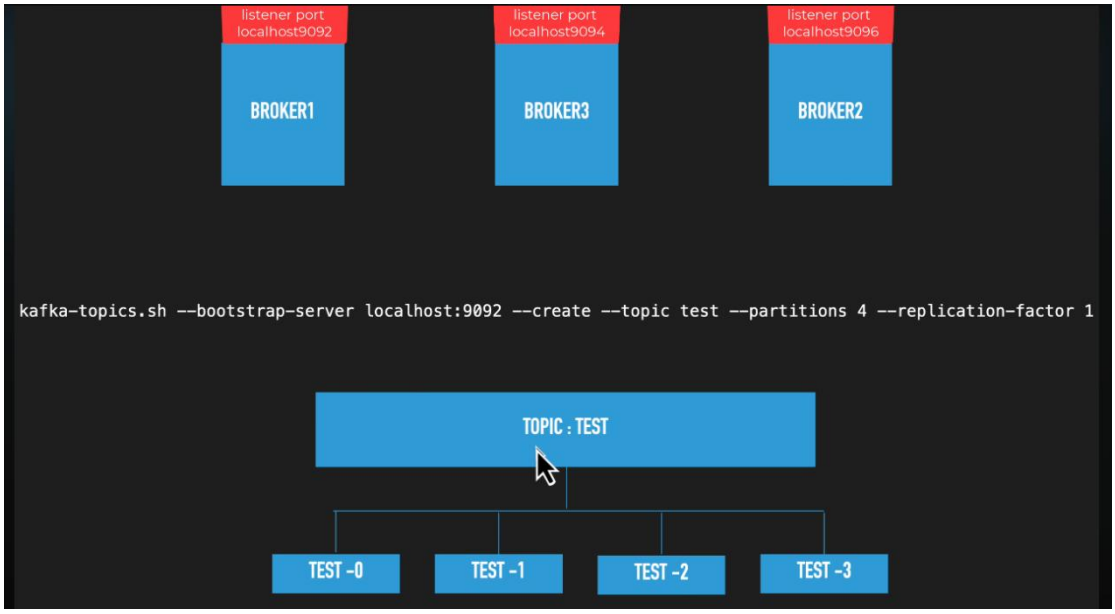


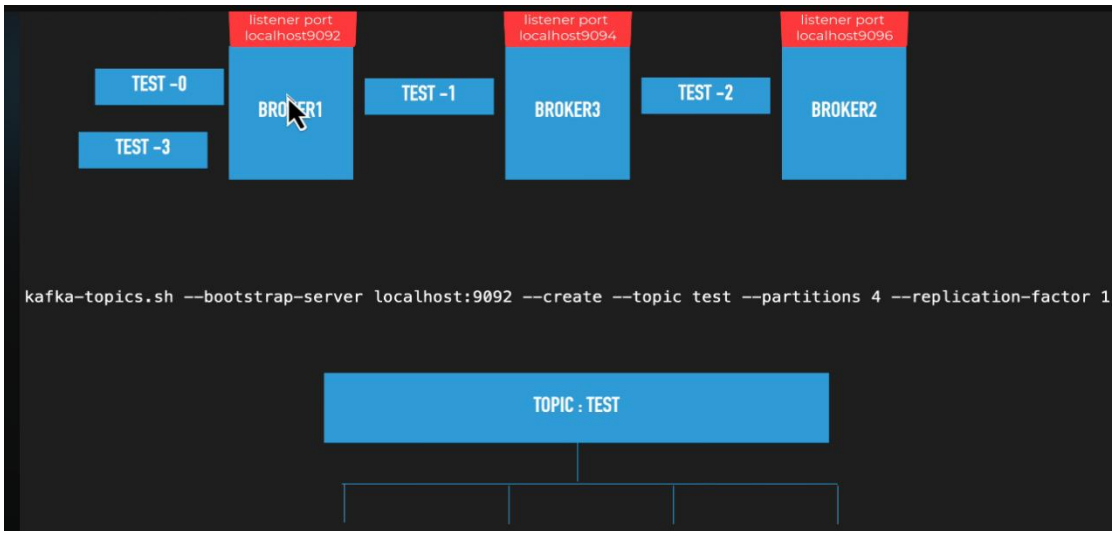
Topic name: TEST

TEST has 4 Partions named: TEST-0, TEST-1, TEST-2, TEST-3

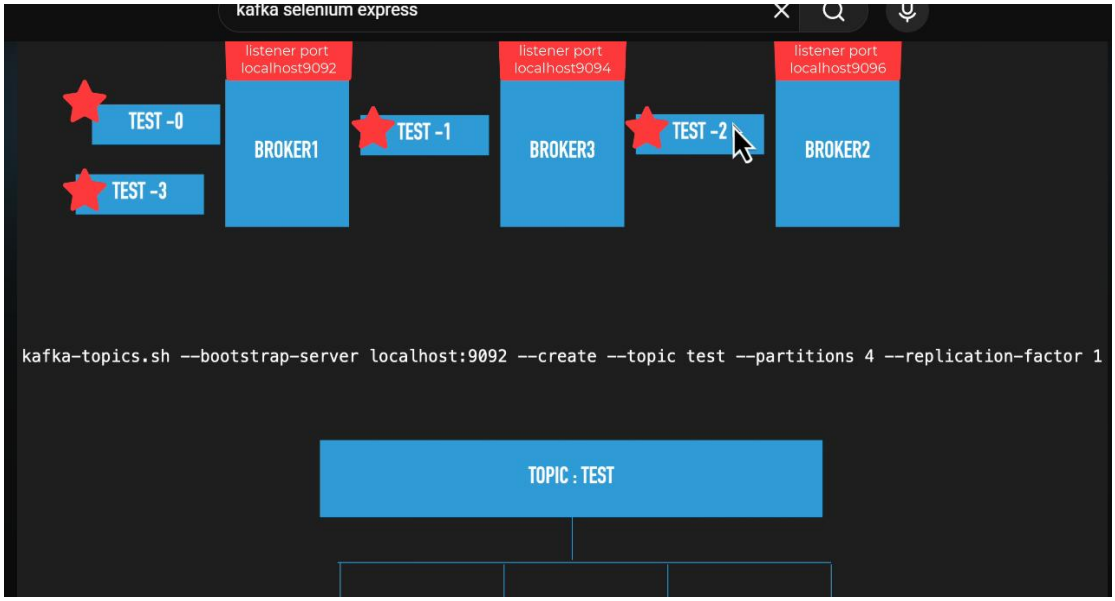
- TEST-0 => TEST is Topic and 0 means 1st partition
- TEST-1 => TEST is Topic and 1 means 2nd partition
- TEST-2 => TEST is Topic and 2 means 3rd partition
- TEST-3 => TEST is Topic and 3 means 4th partition

--replication-factor is 1 means each partition has only 1 copy

Partions assigned to avilable brokers in round-robin manner (sequentially)

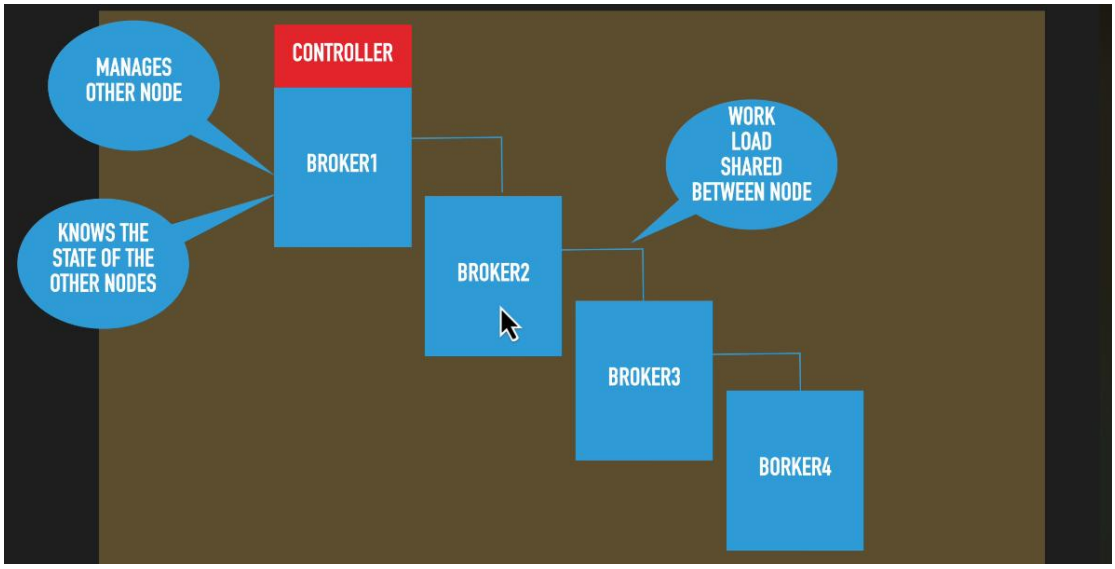


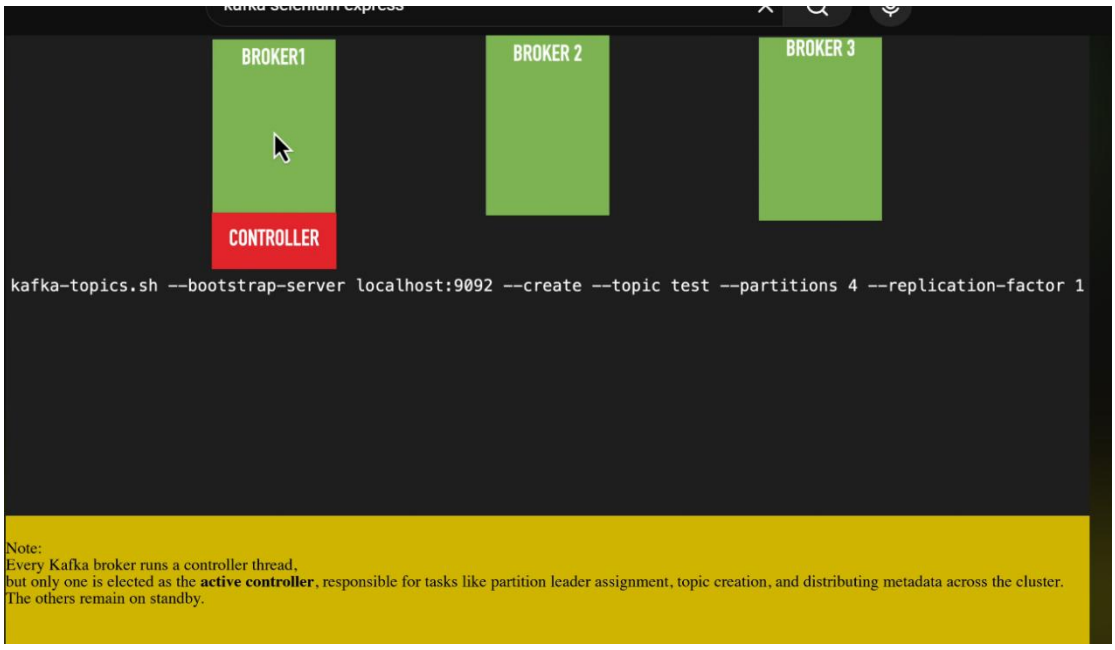
Each partion is leader (in below image red star represnts a leader-partition) because --replication-factor is 1. If --replication-factor is more than 1 then one of them will be elected as leader and rest other replicas of the same partions will be followers.



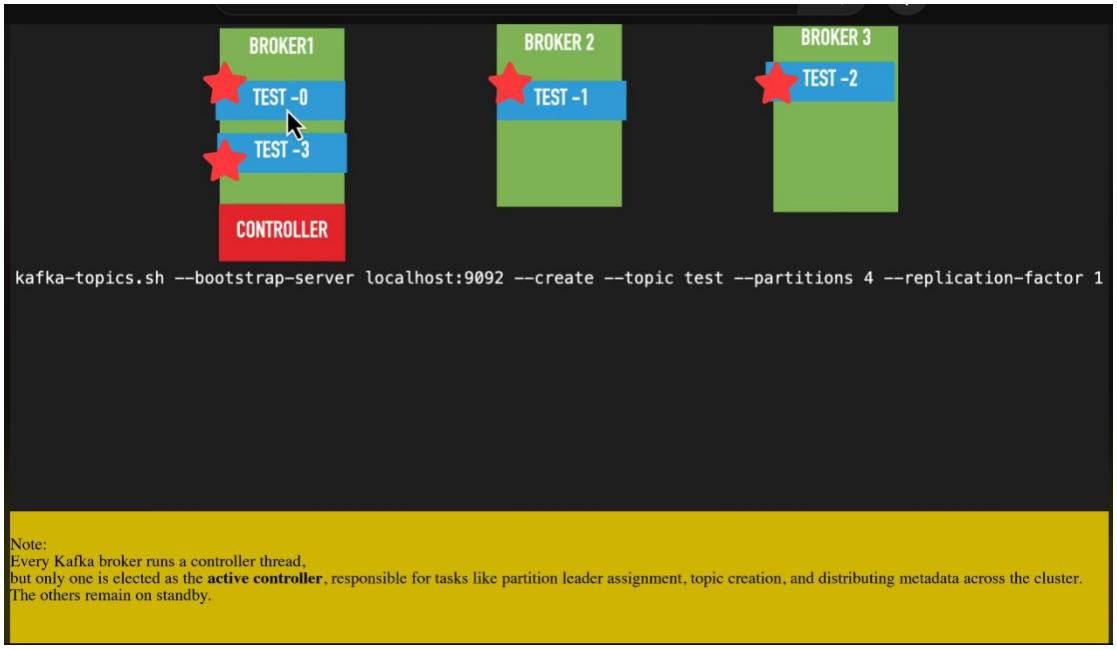
KAFKA - CONTROLLER

In a Kafka cluster we have multiple broker (Server / machine that has kafka software installed), onlly one of the broker will be elected as Controller and rest other broker considered as follower brokers.



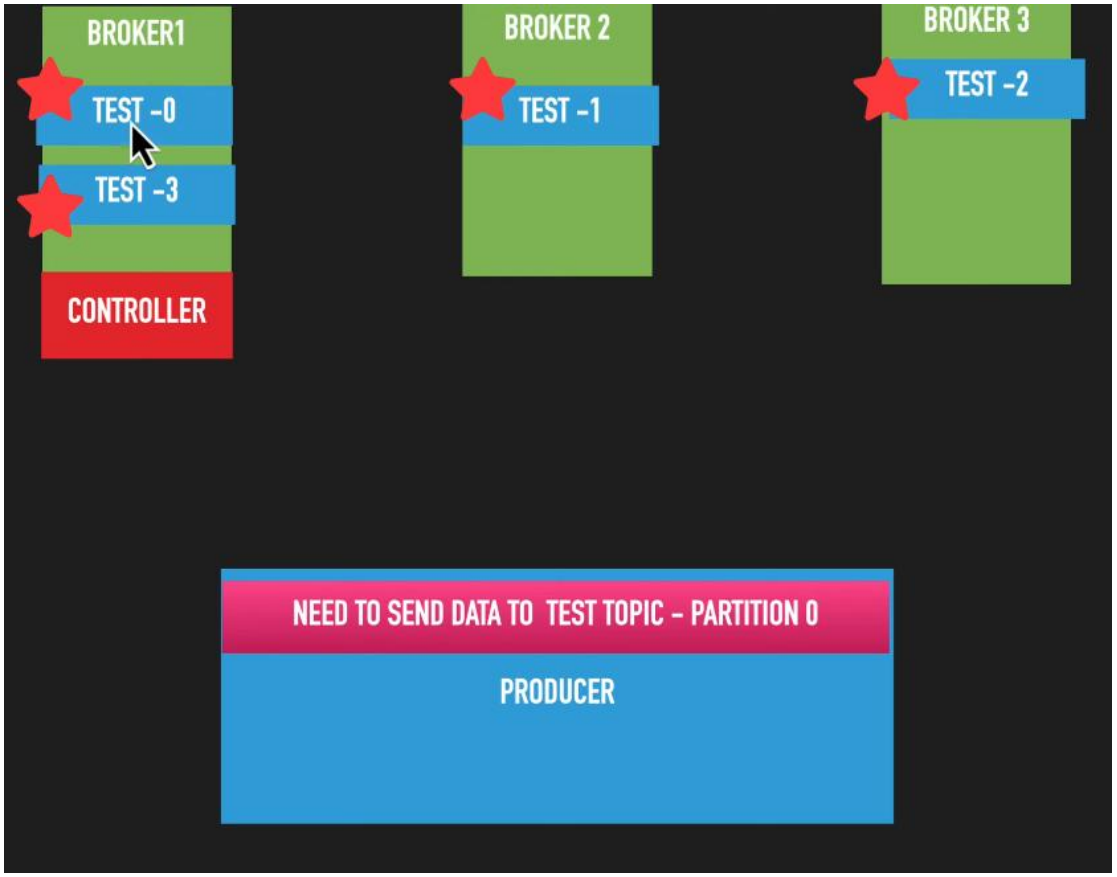


Controller-Broker has the responsibility to create Topic and distributing partitions of the same across available brokers in round-robin manner.

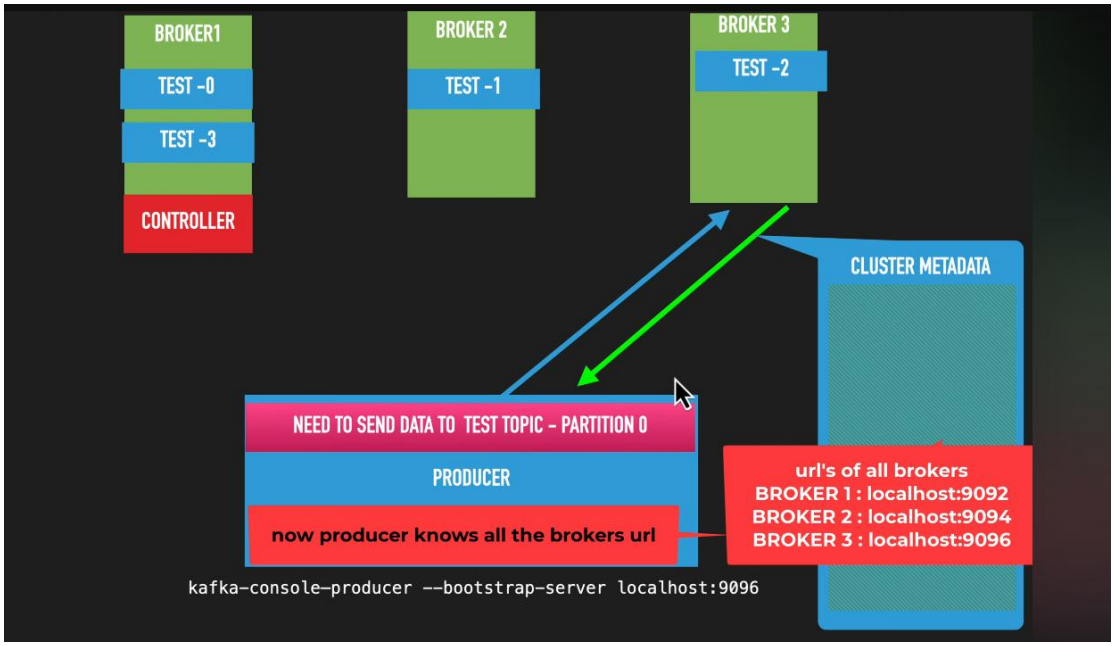


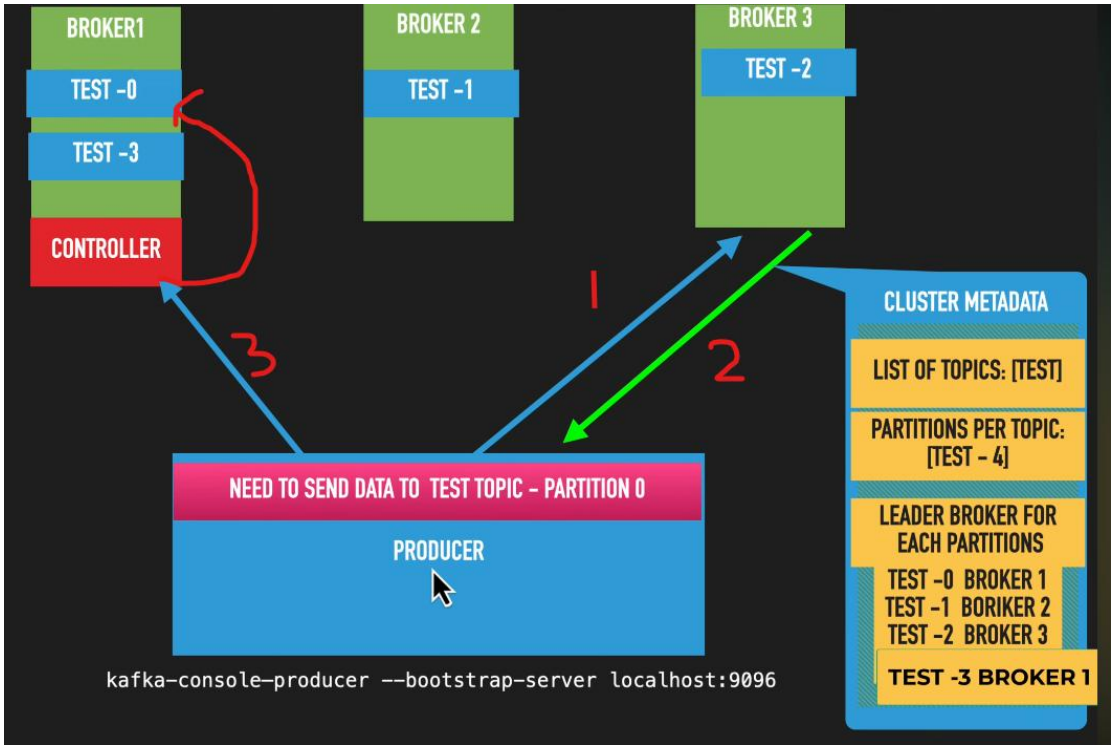
Producer wants to send data to Partition 0 (1st Partition) of the Test topic.

But how Producer know where is the TEST-0 (Partition 0 of the Test topic).
Note: Producer(s) always send data to the leader partition.



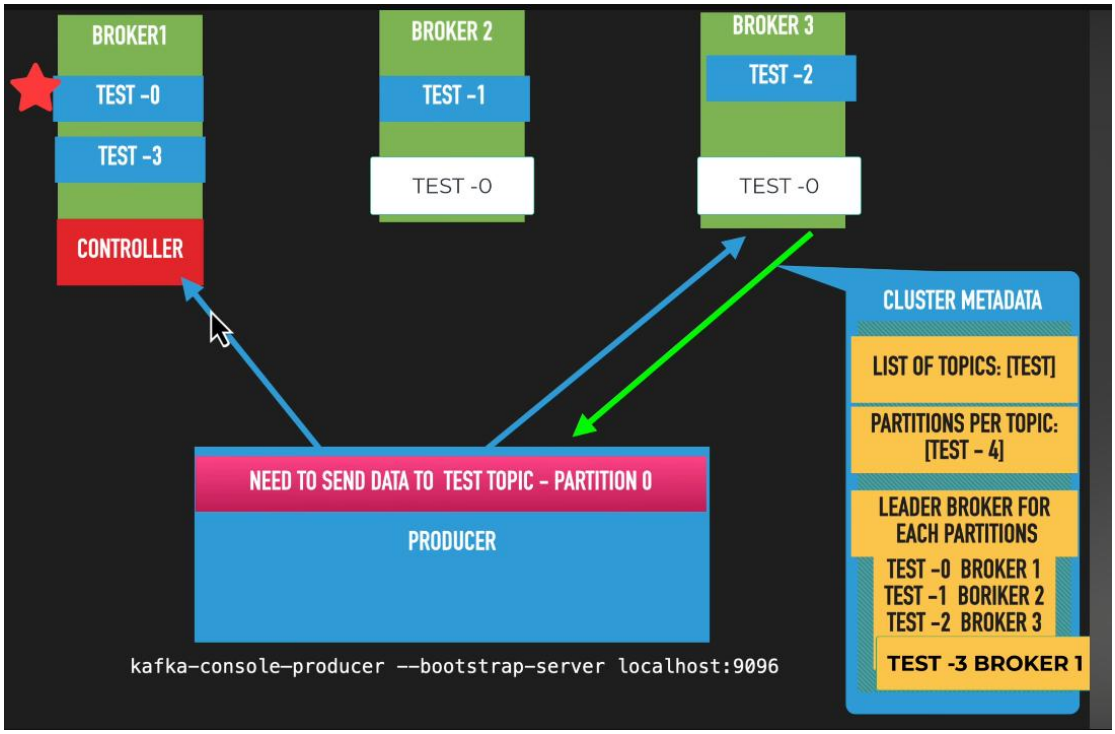
- When we create Producer (kafka-console-producer --bootstrap-server localhost:9096) we define one of the broker address (localhost:9096).
- Producer will send request to that broker which is running at localhost:9096
- And, the same broker which is running at localhost:9096 will return response containing Cluster Metadata





- Cluster Metadata include:
- **List of Topics**: Currently we have only 1 Topic which is TEST
- **Partitions per Topic**: [TEST-4] => TEST Topic has 4 partitions
- **LEADER BROKER FOR EACH PARTITIONS** => it tells Leader partitions are avilable in which brokers.
  - TEST-0 BROKER 1 => TEST-0 leader partition is avilable inside the Broker 1
  - TEST-1 BROKER 1 => TEST-1 leader partition is avilable inside the Broker 2
  - TEST-2 BROKER 1 => TEST-2 leader partition is avilable inside the Broker 3
  - TEST-3 BROKER 1 => TEST-3 leader partition is avilable inside the Broker 1

- Producer always send data to Leader partition, Currently TEST-0 has only 1 replica meaning TEST-0 itself is the leader.
- But, let say TEST-0 has 3 replicas, still producer will send data to Leader partition only.

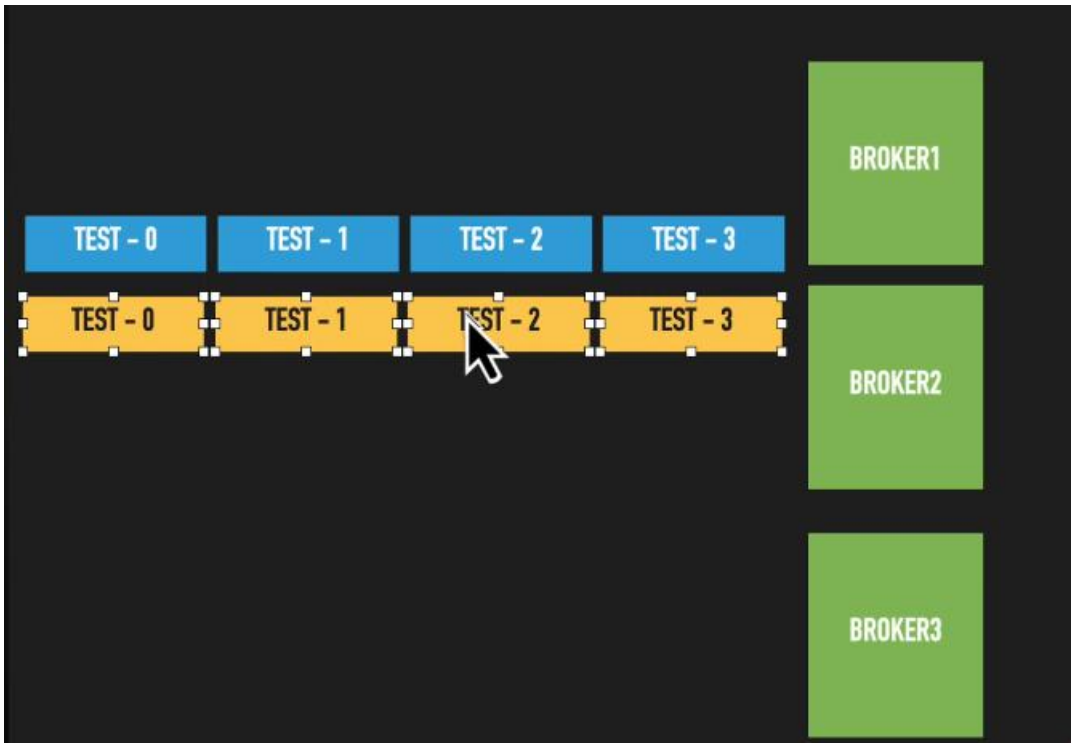


--replication-factor 2

```
kafka-topics.sh --bootstrap-server localhost:9092 --create --topic test --partitions 4 -replication-factor 2
```

- Total partitions = --partitions * --replication-factor = 4 * 2 = 8
- Total leader partitions = --partitions = 4
- Total Follower partitions = Total partitions - Total leader partitions = 8 - 4 = 4

Here, we have 4 partions and each each partion has 2 replicas, meaning one would be leader and another one would be follower



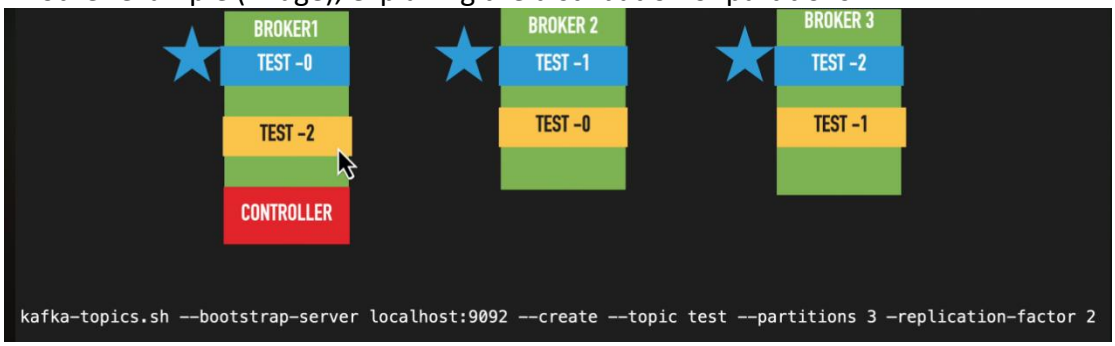
- Blue ones are Leader partitions
- Yellow ones are Follower partions

Round-Robin arrangement

- Broker allocation for partitions happens in **roun-robin** manner and when we start allocating broker to Followers we skip the first broker from which we started allocating brokers to the Leader partitions.
- In simple words, for leader partitions we started with Broker1 so we need to skip the same broker when we going arrange Follwer partitions.
- Again, In very simple words, arrange each partions (leader or follower does not matter) sequentially one after the another but skip prviously started broker.

- Here, we see replicas of the same partitions does not belong to the same broker, they are distributed and that is how kafka achive fault tolerance by distributed replicas of the same partition to different available brokers.

Another example (image), explaning the distribution of partitions.

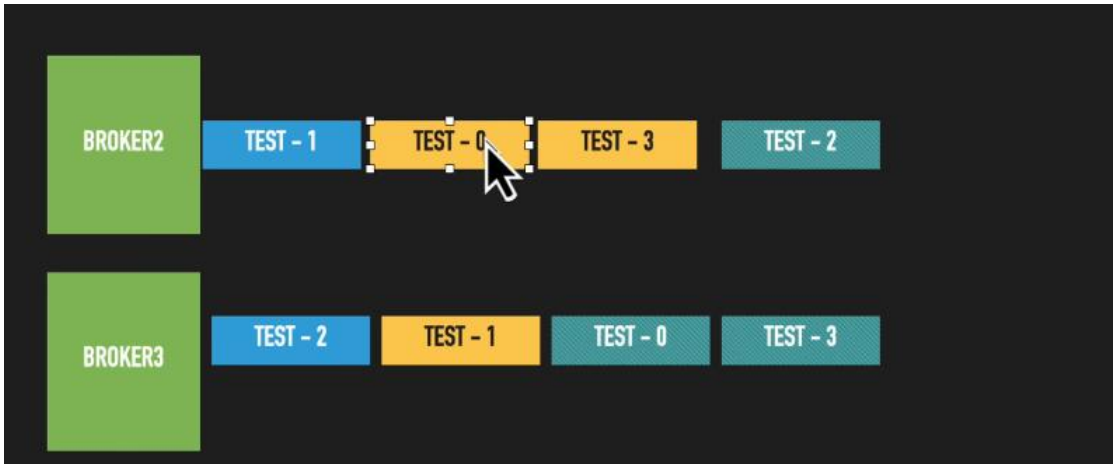


--replication-factor 3

Round-Robin arrangement

Same thing we can observe here, our replicas of the partitions are distributed well across the available Brokers, replicas don't belong to the same broker.

If any broker let say Broker 1 goes down we still all the partions in other Brokers (Broker 1 & Broker 2), achieving fault tolerance.

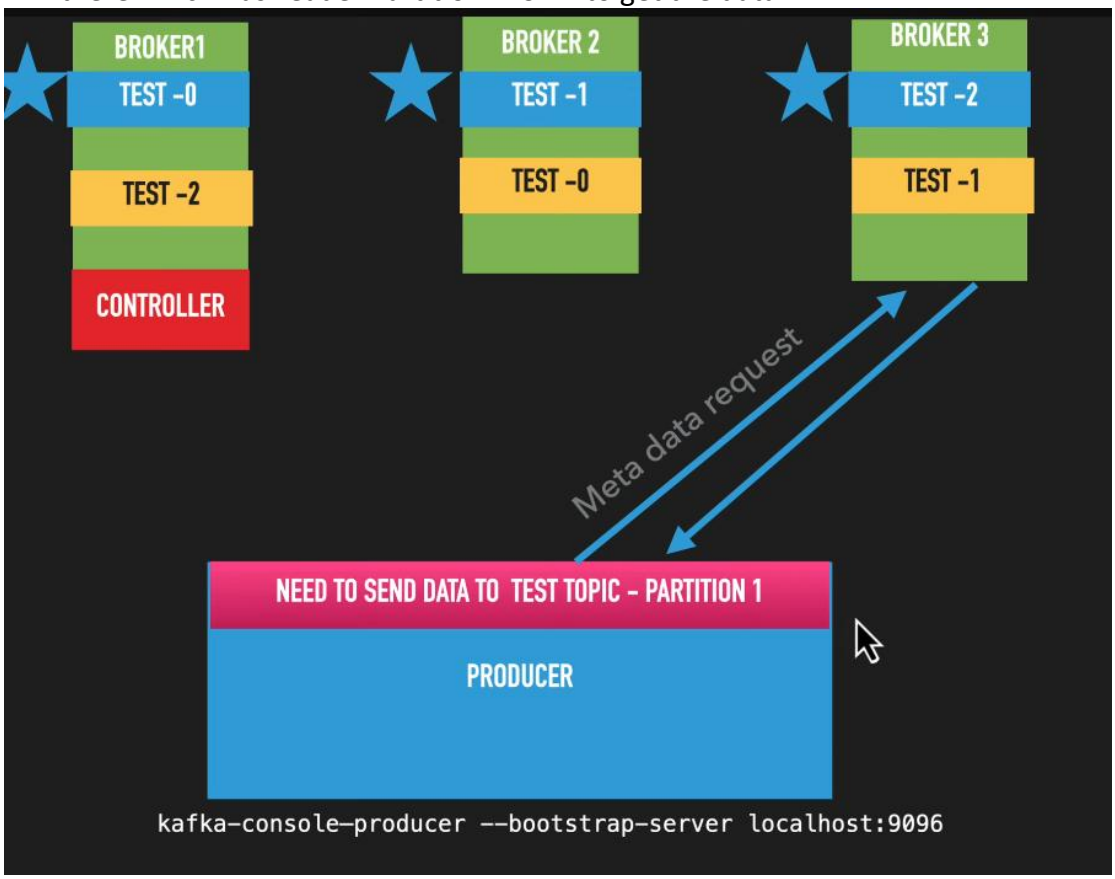


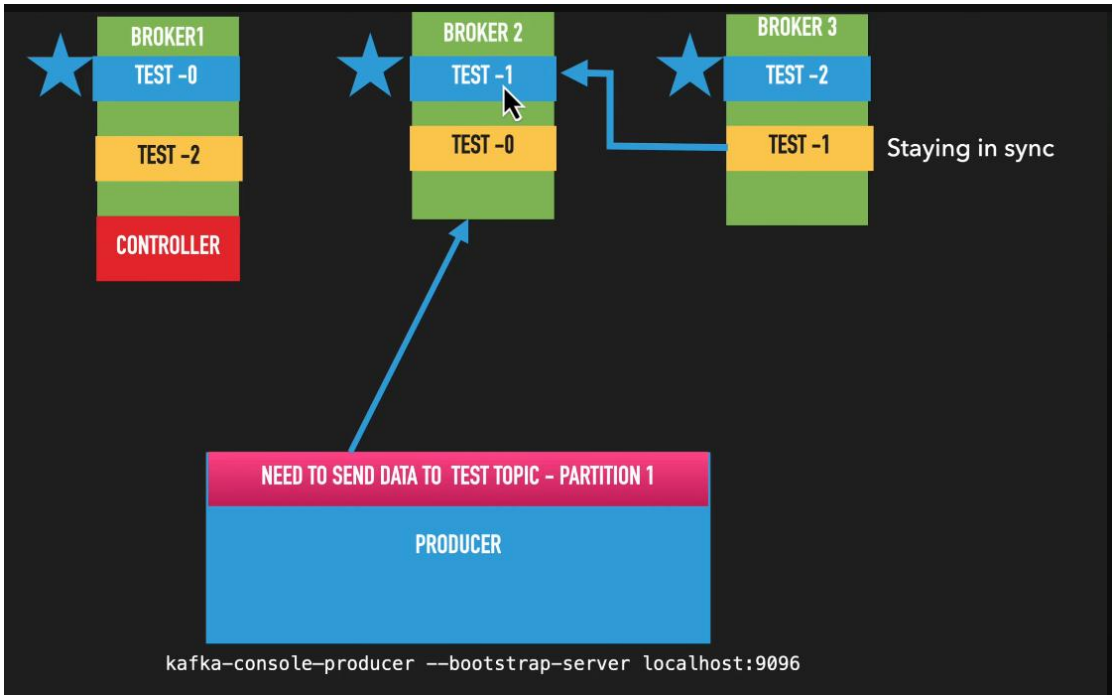
Producer wants to send data to TEST Topic - Partition 1

- Producer wants to send data to TEST Topic - Partition 1 and Follower Partions always copies data from leader partitions because Producer always send data to Leader partitions.

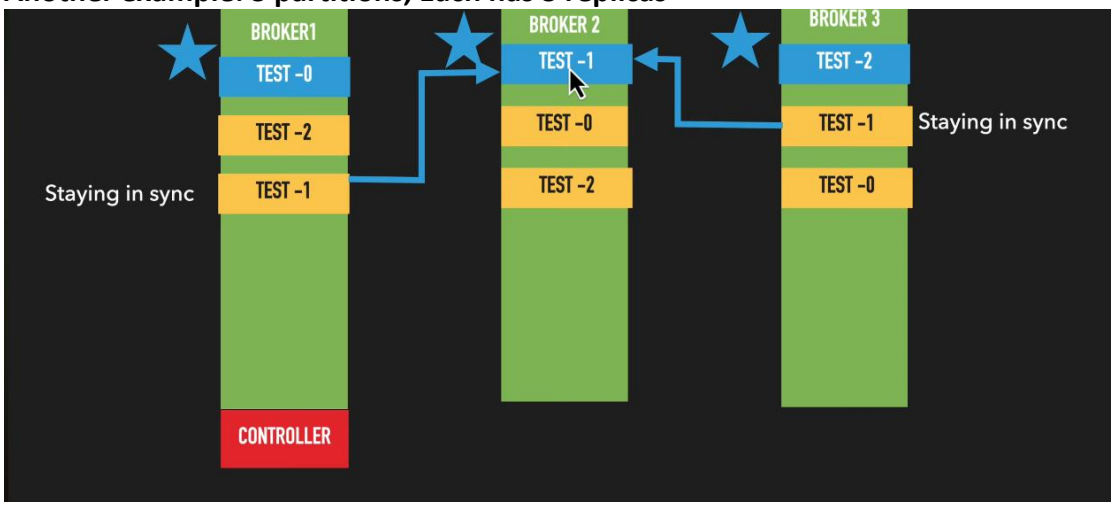
- Follwer partion copies from leader partition bcoz if Broker2 which has Leader partion TEST-1 goes down then still we have Follower partition (TEST-1) in Broker3 to get the data and as Leader partion TEST-1 is no more bcoz Broker2 is down so now Follower partition TEST-1 in Broker3 will become new Leader partition. And, after some time Broker2 comes back then Broker2 TEST-1 partition will become Follower partition.

- And let say Broker3 goes down which has Follower partition TEST-1 still no problem bcoz Broker2 is there which has Leader Partition TEST-1 to get the data.

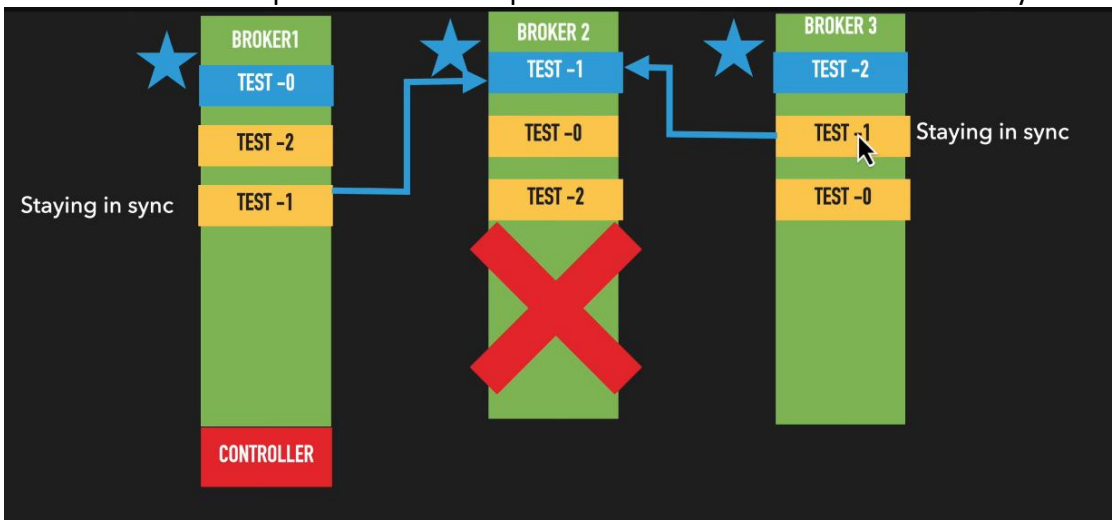




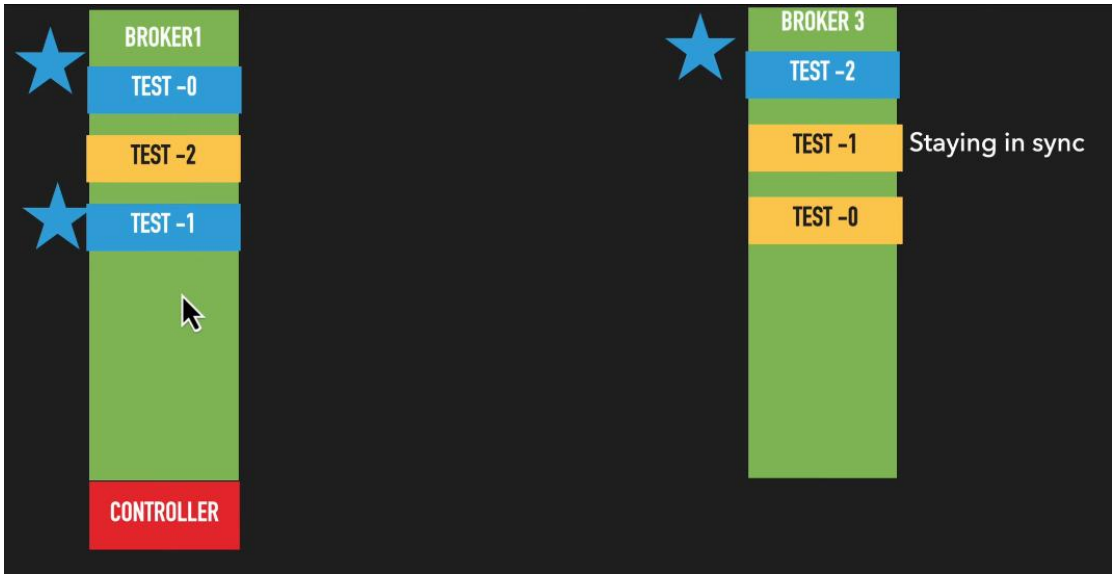
Another example: 3 partitions, Each has 3 replicas



What if Broker2 goes down then who will become new leader, is it TEST-1 Follower partition from Broker1 or TEST-1 Follower partition from Broker3?
Whichever Follower partition is most updated will be elected as new Leader by controller Broker Broker1.



Let say, TEST-1 Follower partition from Broker1 is most updated so Controller Broker Broker1 will select the same as new Leader partition.



ISR (In Sync Replication)

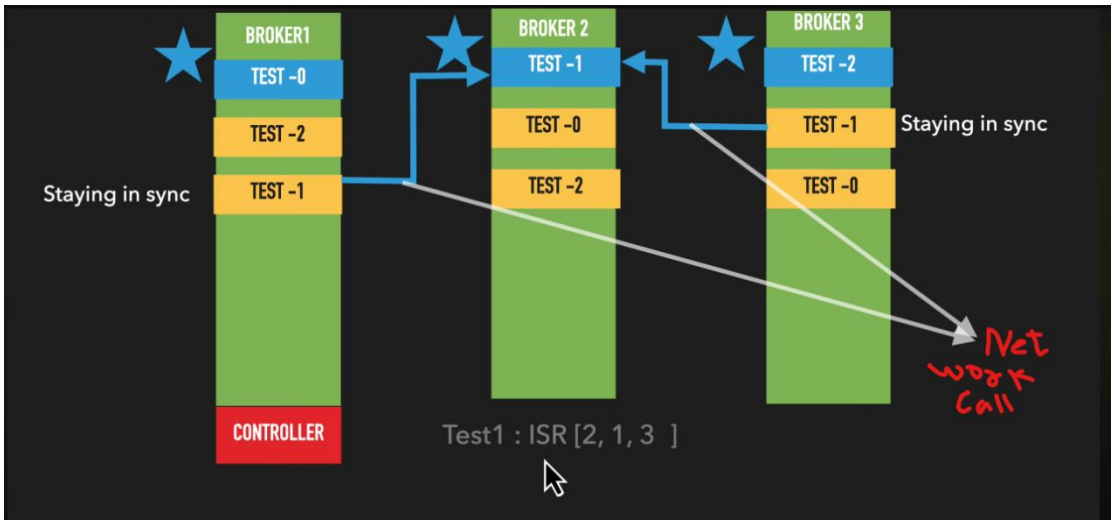
- ISR is a dynamic List.
- ISR managed for Leader Partition within a Broker.
- Leader Partition managed the ISR list.
- ISR list contains Brokers' Id which has either Leader partition (only 1) or Follower Partitions or both having as much data as Leader partion is having.

In the below image, we are seeing Leader partition TEST-1 from BROKER2 ISR list.

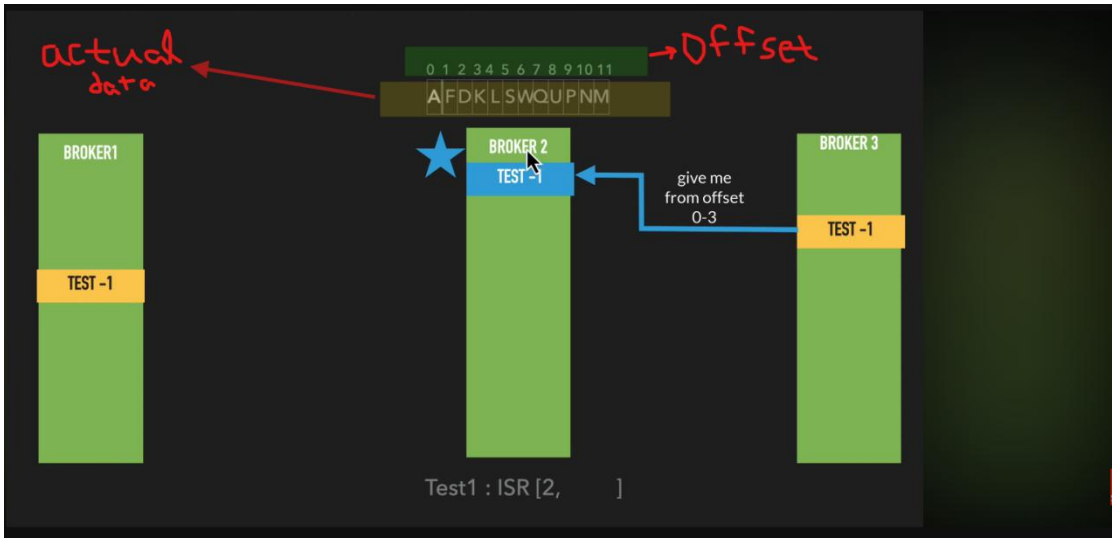
- Follower Partion TEST-1 from BROKER1 make a network call to the LEADER PARTITION TEST-1 of BROKER2 to copy data so that Follower partition TEST-1 from BROKER1 can remain in sync.

- Also, Follower Partion TEST-1 from BROKER3 make a network call to the LEADER PARTITION TEST-1 of BROKER2 to copy data so that Follower partition TEST-1 from BROKER3 can remain in sync.

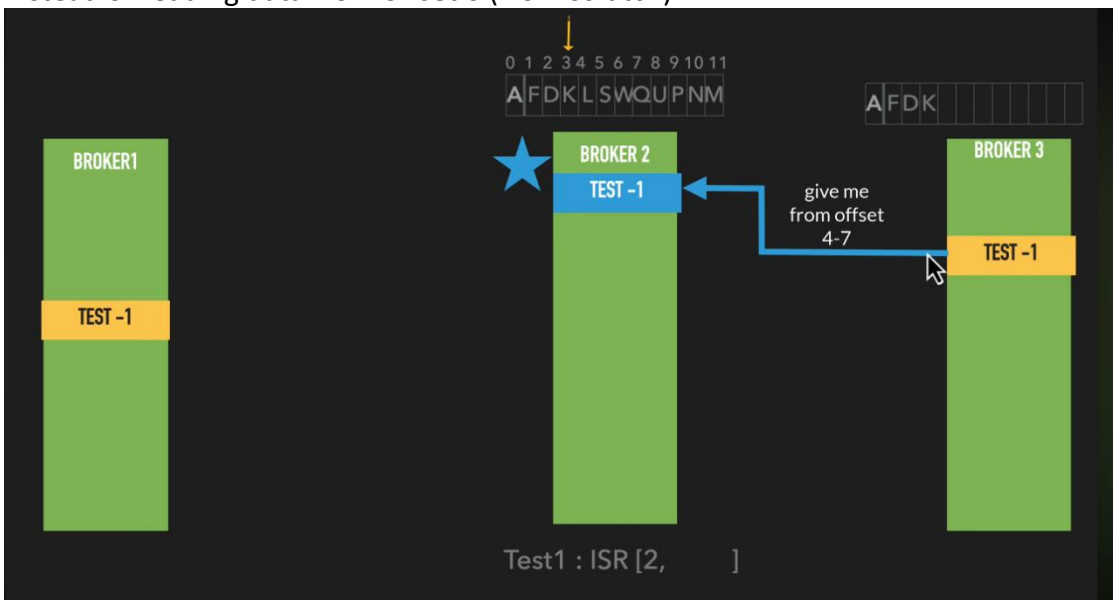
- There is timeout for network call, if data can be copied from Leader to follower in that time period then Follower Partion's Broker Id will be added in the ISR list otherwise can't be added in the ISR list.

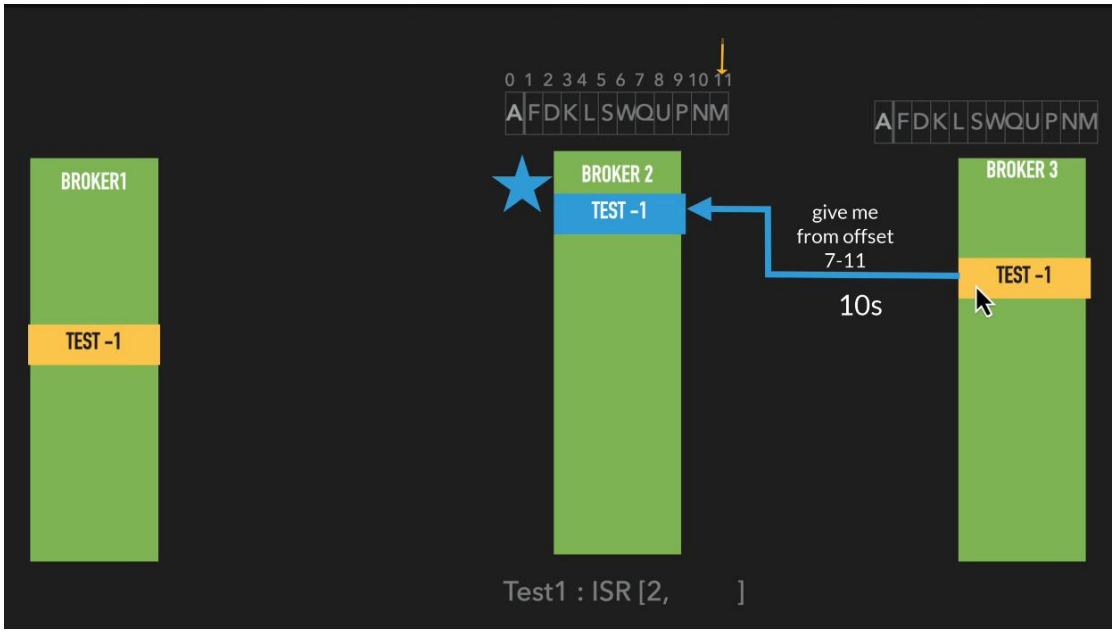


Flow:

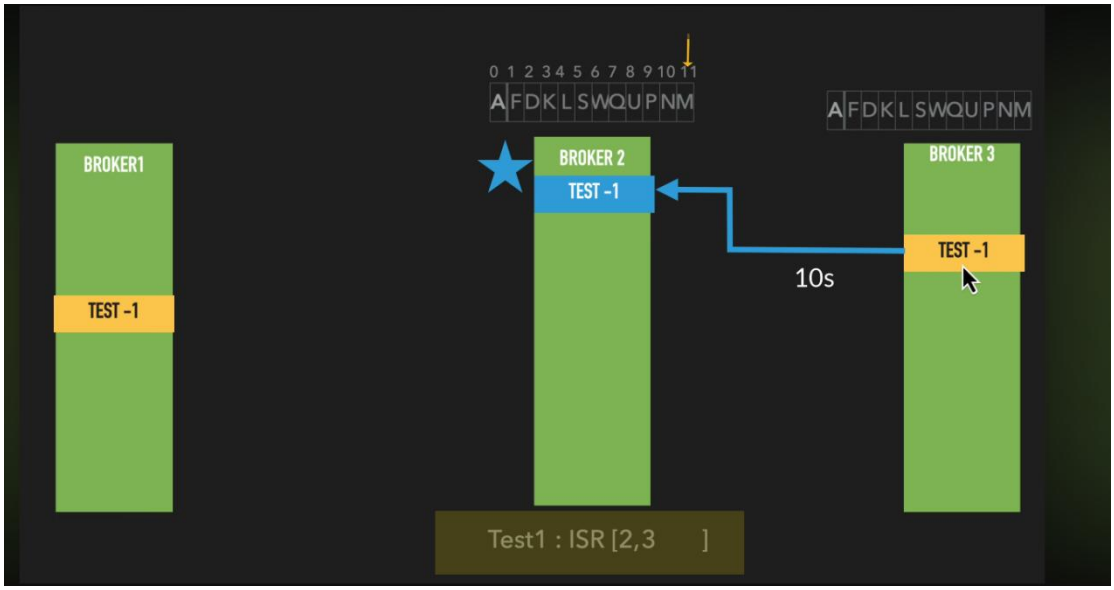


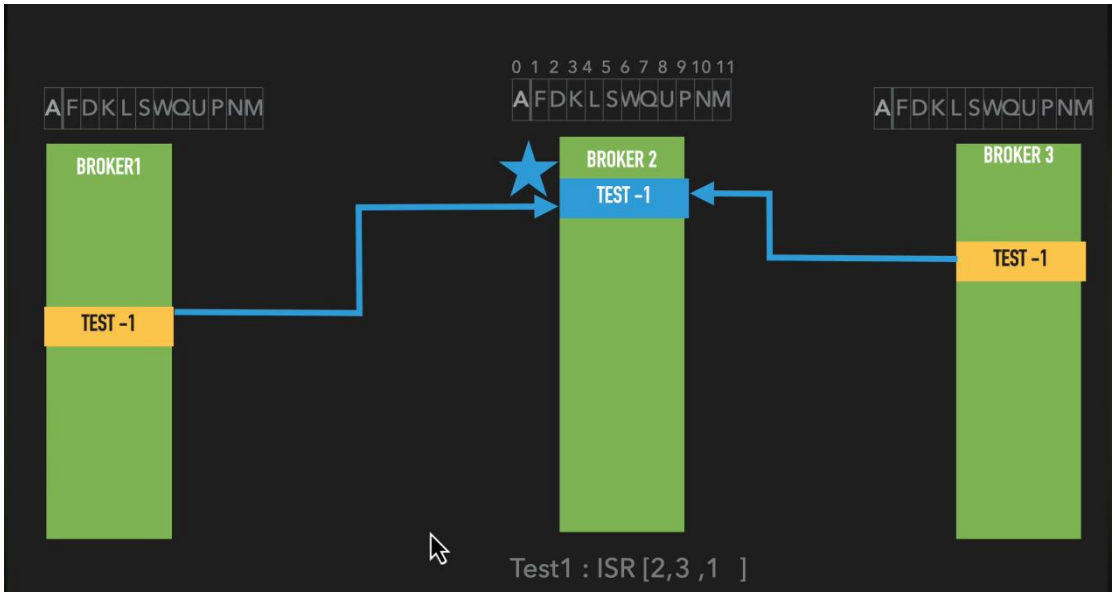
Leader Partition tracks offset counter so that Follower can continue fetching data from last read offset instead of reading data from offset 0 (from scratch).





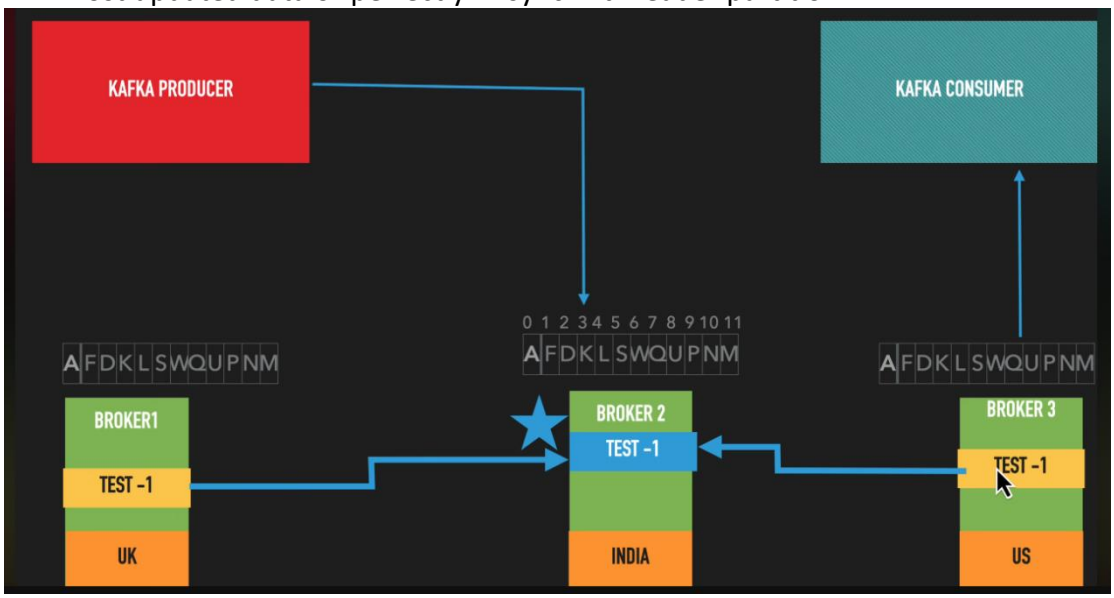
As, Followe Partition TEST-1 from BROKER3 is havng as much data as LEADER PARTION TEST1 from BROKER2 having now 3 (BROKER3) can be added in the ISR list.



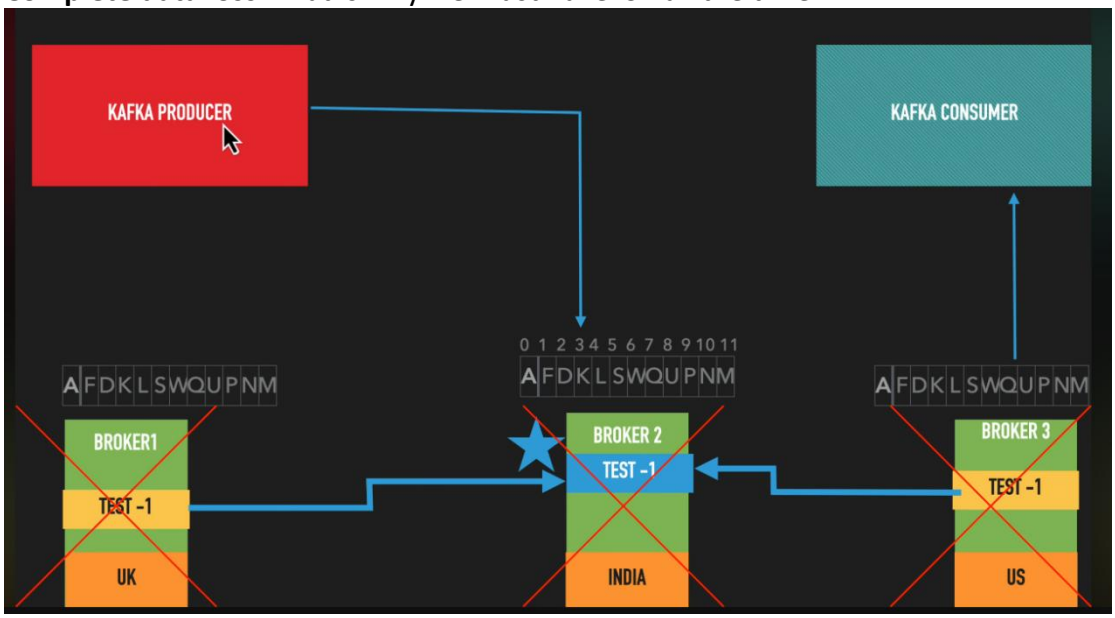


PRODUCER & CONSUMER

- Consumer is the US, so it can be connected to BROKER3 which is in US as well.
- First check the ISR list of the Leader partition, if that ISR list has BRoker3 then we can connect our US Kafka consumer to US Broker (BROKER3), because ISR contains list of those brokers only that have most updated data or perfectly in-sync with leader partition.



Complete data loss: That is why we must have ISR all the time.



🎯 Scenario: Online Food Delivery App

Topic: `orders`

Partitions: **3** → P0, P1, P2

Consumer Group: **OrderProcessors** (3 consumers)

📄 Step 1 — Producers send data

Users place orders from the app.

Producer sends order events:

Order ID	City	Key used	Partition
101	Delhi	Delhi	P0
102	Mumbai	Mumbai	P1
103	Delhi	Delhi	P0
104	Bangalore	Bangalore	P2

👉 Because **city is used as key**, all Delhi orders go to same partition → maintains order sequence.

⚙️ Step 2 — Kafka stores data

Partitions now contain:

```
P0 → Order101, Order103
P1 → Order102
P2 → Order104
```

👥 Step 3 — Consumer group assignment

Consumer Group = **OrderProcessors**

Kafka assigns partitions:

```
Consumer A → P0
Consumer B → P1
Consumer C → P2
```

Rule:

> One partition → one consumer inside a group

Get Plus ×
Save

🔄 Step 4 — Processing flow

Each consumer processes its partition messages sequentially.

Example:

Consumer A processing P0

```
Read Order101 → Validate → Payment → Assign Delivery
Read Order103 → Validate → Payment → Assign Delivery
```

Consumer B processing P1

```
Read Order102 → Process
```

Consumer C processing P2

```
Read Order104 → Process
```

All run **in parallel** → system is fast and scalable.

💥 Step 5 — Failure handling

If **Consumer B crashes**

Kafka reassigns:

```code
Consumer A → P0
Consumer C → P1 + P2
```

No data loss. Processing continues.

🧠 Why this design is powerful

This setup gives:

- parallel processing
- scalability
- ordering within a partition
- fault tolerance

✅ **Real-life summary sentence (interview gold line):**

> Producers distribute messages across partitions based on keys, and consumers in a group divide those partitions among themselves so each partition is processed by exactly one consumer, enabling parallel and reliable processing.